

Project 8: Spring Cloud Config Server CI/CD Demo

Overview

This project demonstrates how to deploy an infrastructure-level microservice (Spring Cloud Config) using Jenkins. The key learning objective is ****Bootstrapping infrastructure components securely****. Config servers typically run on port `8888` and provide centralized configuration to other microservices across the network.

Prerequisites

Ensure the following are installed and running on your Jenkins server:

- Docker & Trivy
- Java 17 & Maven

Step 1: Network Setup

Create a shared Docker network for your infrastructure and deployments:

```
docker network create jenkins-net
```

Step 2: Infrastructure Setup

Run SonarQube and Jenkins on the `jenkins-net` network:

```
# Run SonarQube
```

```
docker run -d --name sonarqube -p 9000:9000 --network jenkins-net  
sonarqube:lts-community
```

```
# Run Jenkins
```

```
docker run -d --name jenkins -p 8080:8080 -p 50000:50000 --network  
jenkins-net -v /var/run/docker.sock:/var/run/docker.sock -v  
jenkins_home:/var/jenkins_home jenkins/jenkins:lts
```

Step 3: Jenkins Configuration

1. Log into Jenkins and ensure Docker Pipeline, SonarQube Scanner, and Pipeline plugins are installed.
2. Configure SonarQube Server in Manage Jenkins -> System.
3. Configure Global Tools (Maven, JDK, Sonar Scanner).

Step 4: Pipeline Execution

1. Create a new Pipeline job in Jenkins.
2. Choose "Pipeline script from SCM" or paste the contents of Jenkinsfile.
3. Run the build.

Step 5: Verification

Access the deployed Config Server's REST API to fetch the sample configuration we baked in:

- URL: `http://localhost:8888/sample-client/dev``

- Expected Output: A JSON payload containing ``"message": "Hello from the Jenkins deployed Config Server!"``